

# **CSA0613 - Design and Analysis of Algorithms**

## **CO4-AT2-Industry-based Problem Solving**

**Divyesh S P (192424147).**

### **Logistics Route Optimization – Travelling Salesman Problem**

#### **1. Aim**

To implement the **Travelling Salesman Problem (TSP)** using the **Held–Karp Dynamic Programming algorithm** and find the minimum-cost route that visits all 18 cities exactly once and returns to the starting city.

#### **2. Problem Statement**

A logistics company needs to deliver goods to **18 cities**. The vehicle must:

- Start from the origin city.
- Visit every city exactly once.
- Return to the origin.
- Minimize the total transportation cost.

This is a **Travelling Salesman Problem (TSP)**.

#### **3. Method Used – Held-Karp Algorithm**

The Held-Karp algorithm solves TSP using **Dynamic Programming**.

## State Representation

Let:

$DP[S][j]$

represent the minimum cost of starting from city 0, visiting all cities in set  $S$ , and ending at city  $j$ .

## Recurrence

$$DP[S][j] = \min_{i \in S, i \neq j} \{DP[S - \{j\}][i] + \text{distance}[i][j]\}$$
$$DP[S][j] = \min_{i \in S, i \neq j} \{DP[S - \{j\}][i] + \text{distance}[i][j]\}$$

Finally, return to the starting city:

$$\text{Cost} = \min_j (DP[\text{All}][j] + \text{distance}[j][0])$$

## 4. Constraints

- Number of cities: **18**
- Distance matrix: **18 × 18**
- Distance from a city to itself: **0**
- Every city must be visited exactly once.
- The route must return to the origin.
- The distance/cost between every pair of cities is required.

The major limitation is exponential growth.

## Time Complexity

$$O(2^{2n})$$

## Space Complexity

$$O(2^n)$$

For 18 cities, the DP contains approximately:

$$18 \times 2^{18}$$

possible state entries.

This is feasible for a small assessment program, but becomes expensive as the number of cities increases.

## 5. Python Implementation

This code is suitable for **Google Colab, Jupyter Notebook, or Python IDLE.**

```
INF = float('inf')

# 18 x 18 distance matrix
distance = [
    [0, 10, 15, 20, 12, 18, 25, 30, 22, 14, 28, 16, 24, 19, 27, 21, 13, 17],
    [10, 0, 35, 25, 17, 22, 30, 20, 28, 24, 32, 18, 26, 29, 21, 31, 23, 27],
    [15, 35, 0, 30, 20, 28, 18, 25, 22, 32, 26, 24, 16, 27, 19, 29, 21, 23],
    [20, 25, 30, 0, 15, 17, 27, 22, 18, 26, 20, 28, 24, 19, 31, 23, 21, 29],
    [12, 17, 20, 15, 0, 16, 24, 21, 19, 23, 27, 25, 18, 22, 26, 20, 14, 28],
    [18, 22, 28, 17, 16, 0, 19, 26, 24, 20, 22, 30, 21, 25, 29, 23, 27, 15],
    [25, 30, 18, 27, 24, 19, 0, 17, 23, 28, 20, 22, 26, 18, 24, 30, 21, 25],
    [30, 20, 25, 22, 21, 26, 17, 0, 16, 24, 28, 19, 23, 27, 21, 25, 29, 18],
    [22, 28, 22, 18, 19, 24, 23, 16, 0, 21, 25, 17, 20, 26, 28, 22, 24, 19],
    [14, 24, 32, 26, 23, 20, 28, 24, 21, 0, 18, 26, 22, 25, 27, 19, 21, 23],
    [28, 32, 26, 20, 27, 22, 20, 28, 25, 18, 0, 24, 29, 21, 23, 27, 31, 19],
    [16, 18, 24, 28, 25, 30, 22, 19, 17, 26, 24, 0, 21, 23, 27, 20, 26, 22],
    [24, 26, 16, 24, 18, 21, 26, 23, 20, 22, 29, 21, 0, 28, 24, 26, 18, 25],
    [19, 29, 27, 19, 22, 25, 18, 27, 26, 25, 21, 23, 28, 0, 20, 24, 26, 22],
    [27, 21, 19, 31, 26, 29, 24, 21, 28, 27, 23, 27, 24, 20, 0, 18, 25, 23],
    [21, 31, 29, 23, 20, 23, 30, 25, 22, 19, 27, 20, 26, 24, 18, 0, 17, 21],
    [13, 23, 21, 21, 14, 27, 21, 29, 24, 21, 31, 26, 18, 26, 25, 17, 0, 16],
    [17, 27, 23, 29, 28, 15, 25, 18, 19, 23, 19, 22, 25, 22, 23, 21, 16, 0]
]
```

```

n = len(distance)

# DP dictionary
dp = {}
parent = {}

# Starting from city 0
for city in range(1, n):
    mask = (1 << 0) | (1 << city)
    dp[(mask, city)] = distance[0][city]
    parent[(mask, city)] = 0

# Held-Karp Dynamic Programming
for size in range(3, n + 1):

    for mask in range(1 << n):

        if not (mask & 1):
            continue

        if mask.bit_count() != size:
            continue

        for last in range(1, n):

            if not (mask & (1 << last)):
                continue

            previous_mask = mask ^ (1 << last)

            best_cost = INF
            best_city = -1

            for previous in range(1, n):

                if not (previous_mask & (1 << previous)):
                    continue

                key = (previous_mask, previous)

                if key in dp:
                    cost = dp[key] + distance[previous][last]

                    if cost < best_cost:

```

```

        best_cost = cost
        best_city = previous

    if best_city != -1:
        dp[(mask, last)] = best_cost
        parent[(mask, last)] = best_city

# Complete set of cities
full_mask = (1 << n) - 1

# Find minimum tour cost
best_cost = INF
last_city = -1

for city in range(1, n):
    cost = dp[(full_mask, city)] + distance[city][0]

    if cost < best_cost:
        best_cost = cost
        last_city = city

# Reconstruct route
route = []
mask = full_mask
city = last_city

while city != 0:
    route.append(city)
    previous = parent[(mask, city)]
    mask = mask ^ (1 << city)
    city = previous

route.append(0)
route.reverse()
route.append(0)

print("===== TSP RESULT =====")
print("Number of cities:", n)
print("Optimal Route:", " -> ".join(map(str, route)))
print("Minimum Transportation Cost:", best_cost)

```

## 6. Output

```
===== TSP RESULT =====
```

```
Number of cities: 18
```

```
Optimal Route: 0 -> 4 -> 16 -> 12 -> 2 -> 14 -> 15 -> 9 -> 10 -> 17 -> 5 -> 3 -> 13 -> 6 -> 7 -> 8 -> 11 -> 1 -> 0
```

```
Minimum Transportation Cost: 300
```

## 7. How the Program Works

The program starts from **City 0**.

It calculates the cheapest way to reach every city while remembering which cities have already been visited.

For example:

$0 \rightarrow 3$

Then it considers:

$0 \rightarrow 3 \rightarrow 5$

Then:

$0 \rightarrow 3 \rightarrow 5 \rightarrow 7$

The DP table stores these partial solutions so that the same calculations don't have to be repeated.

Finally, it adds the cost of returning from the last city to **City 0**.

## 8. Real-World Feasibility

The Held-Karp algorithm can provide an **exact optimal solution**, making it useful for small logistics problems.

However, it becomes difficult for large numbers of cities because both time and memory requirements grow exponentially.

For real logistics systems with hundreds or thousands of locations, methods such as:

- Genetic Algorithms
- Ant Colony Optimization
- Simulated Annealing
- Nearest Neighbor
- Integer Programming
- Vehicle Routing Problem (VRP) algorithms

are often more practical.

## 9. Scalability

| Cities    | Difficulty                          |
|-----------|-------------------------------------|
| 10        | Easy                                |
| 15        | Manageable                          |
| <b>18</b> | <b>Suitable for this assessment</b> |
| 20        | More memory/time                    |

| Cities | Difficulty                                |
|--------|-------------------------------------------|
| 25     | Very expensive                            |
| 30+    | Generally impractical for exact Held-Karp |

The major scalability problem is:

$$O(n^2 2^n)$$

As  $n$  increases, the number of DP states increases exponentially.

## 10. Result

The Travelling Salesman Problem for **18 cities** was successfully implemented using the **Held-Karp Dynamic Programming algorithm**. The algorithm calculates the minimum transportation cost by considering subsets of cities and finding the optimal route that visits every city exactly once and returns to the origin. The implementation demonstrates that Dynamic Programming can solve small and medium-sized TSP instances exactly, while exponential growth limits its use for large real-world logistics networks.